DS H.W SET 8

1. Topological Sorting (Using DFS)

```
#include <stdio.h>
#define MAX 100
int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int vertices;
 void push(int vertex)
{
stack[++top] = vertex;
}
void DFS(int vertex)
{
visited[vertex] = 1;

for(int i = 0; i < vertices; i++)
{
if(graph[vertex][i] == 1 && !visited[i])
{
DFS(i);
}
}

push(vertex);
}
void topologicalSort()
{
for(int i = 0; i < vertices; i++)
{
visited[i] = 0;
}
```

```
for(int i = 0; i < vertices; i++)
{
if(!visited[i])
{
DFS(i);
}
}
printf("\nTopological Order:\n");
while(top != -1)
{
printf("%d ", stack[top--]);
}
printf("\n");
}
int main()
{
int edges, u, v;
printf("Enter number of vertices: ");
scanf("%d", &vertices);
printf("Enter number of edges: ");
scanf("%d", &edges);
for(int i = 0; i < vertices; i++)
{
for(int j = 0; j < vertices; j++)
{
graph[i][j] = 0;
}
}
printf("\nEnter the directed edges (u v):\n");
for(int i = 0; i < edges; i++)
{
scanf("%d %d", &u, &v);
```

```
graph[u][v] = 1;

}

topologicalSort();

return 0;

}
```

Output

```
Enter number of vertices: 6

Enter number of edges: 6

Enter the directed edges (u v):

5 2

5 0

4 0

4 1

2 3

3 1
```

2. Kruskal's Algorithm

```
#include <stdio.h>
#define MAX 100
struct Edge {
int src, dest, weight;
};
int parent[MAX];
int find(int i)
{
while(parent[i] != i)
i = parent[i];
return i;
}
void Union(int a, int b)
{
int rootA = find(a);
int rootB = find(b);
parent[rootA] = rootB;
}
void sortEdges(struct Edge edges[], int E)
{
struct Edge temp;

for(int i = 0; i < E - 1; i++)
{
for(int j = 0; j < E - i - 1; j++)
{
if(edges[j].weight > edges[j + 1].weight)
{
temp = edges[j];
edges[j] = edges[j + 1];
edges[j + 1] = temp;
}
```

```
}
}
}
int main()
{
int V, E;
printf("Enter Number of Vertices: ");
scanf("%d", &V);
printf("Enter Number of Edges: ");
scanf("%d", &E);
struct Edge edges[E];
printf("\nEnter Source Destination Weight:\n");
for(int i = 0; i < E; i++)
{
scanf("%d%d%d",
&edges[i].src,
&edges[i].dest,
&edges[i].weight);
}
for(int i = 0; i < V; i++)
{
parent[i] = i;
}
sortEdges(edges, E);
int mstCost = 0;
int edgeCount = 0;
printf("\nEdges in MST:\n");
for(int i = 0; i < E && edgeCount < V - 1; i++)
{
int u = edges[i].src;
int v = edges[i].dest;
int setU = find(u);
int setV = find(v);
```

```
if(setU != setV)
{
printf("%d -- %d : %d\n",
u, v, edges[i].weight);

mstCost += edges[i].weight;
Union(setU, setV);
edgeCount++;
}
}
printf("\nTotal Cost of MST = %d\n", mstCost);
return 0;
}
```

Output

```
Enter Number of Vertices: 5
Enter Number of Edges: 7
Enter Source Destination Weight:
0 1 1
0 2 5
1 2 3
1 3 4
2 4 6
3 4 2
2 3 5
Edges in MST:
0 -- 1 : 1
3 -- 4 : 2
1 -- 2 : 3
1 -- 3 : 4
Total Cost of MST = 10
```

Prim's Algorithm

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main()
{
int n;
int graph[MAX][MAX];

printf("Enter number of vertices: ");
scanf("%d", &n);

printf("Enter adjacency matrix:\n");

for(int i = 0; i < n; i++)
{
for(int j = 0; j < n; j++)
{
scanf("%d", &graph[i][j]);
}
}

int selected[MAX] = {0};
selected[0] = 1;

int edges = 0;
int cost = 0;

printf("\nEdges in MST:\n");

while(edges < n - 1)
```

```
{
int min = INT_MAX;
int x = 0, y = 0;

for(int i = 0; i < n; i++)
{
if(selected[i])
{
for(int j = 0; j < n; j++)
{
if(!selected[j] &&
graph[i][j] &&
graph[i][j] < min)
{
min = graph[i][j];
x = i;
y = j;
}
}
}
}

printf("%d - %d : %d\n", x, y, graph[x][y]);

cost += graph[x][y];
selected[y] = 1;
edges++;
}

printf("\nTotal Cost of MST = %d\n", cost);

return 0;
}
```

Output

Enter number of vertices: 5

Enter adjacency matrix:

0 1 5 0 0

1 0 3 4 0

5 3 0 5 6

0 4 5 0 2

0 0 6 2 0

Edges in MST:

0 - 1 : 1

1 - 2 : 3

1 - 3 : 4

3 - 4 : 2

Total Cost of MST = 10

3 . Minimum Spanning Tree using Prim's Algorithm

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main()
{
int n;
int graph[MAX][MAX];

printf("Enter the number of vertices: ");
scanf("%d", &n);

printf("Enter the adjacency matrix:\n");
printf("(Enter 0 if no edge exists)\n");

for(int i = 0; i < n; i++)
{
```

```
for(int j = 0; j < n; j++)
{
scanf("%d", &graph[i][j]);
}
}

int selected[MAX];

for(int i = 0; i < n; i++)
selected[i] = 0;

selected[0] = 1; // Start from vertex 0

int edgeCount = 0;
int totalCost = 0;

printf("\nEdges in the Minimum Spanning Tree:\n");

while(edgeCount < n - 1)
{
int min = INT_MAX;
int x = -1, y = -1;

for(int i = 0; i < n; i++)
{
if(selected[i])
{
for(int j = 0; j < n; j++)
{
if(!selected[j] &&
graph[i][j] &&
graph[i][j] < min)
{
```

```
min = graph[i][j];
x = i;
y = j;
}
}
}
}

printf("%d --> %d Weight = %d\n", x, y, graph[x][y]);

totalCost += graph[x][y];
selected[y] = 1;
edgeCount++;
}

printf("\nTotal Cost of MST = %d\n", totalCost);

return 0;
}
```

Output

```
Enter the number of vertices: 4
Enter the adjacency matrix:
0 4 5 0
4 0 3 6
5 3 0 2
0 6 2 0
Edges in the Minimum Spanning Tree:
0 --> 1  Weight = 4
1 --> 2  Weight = 3
2 --> 3  Weight = 2
Total Cost of MST = 9
```

4 . Dijkstra's Algorithm

Code

```
#include <stdio.h>
#include <limits.h>

#define MAX 100
 int minDistance(int dist[], int visited[], int n)
{
int min = INT_MAX, min_index = -1;

for(int v = 0; v < n; v++)
{
if(!visited[v] && dist[v] < min)
{
min = dist[v];
min_index = v;
}
}

return min_index;
}
 void dijkstra(int graph[MAX][MAX], int n, int source)
{
int dist[MAX];
int visited[MAX];

// Initialize distances
for(int i = 0; i < n; i++)
{
dist[i] = INT_MAX;
visited[i] = 0;
}
```

```
dist[source] = 0;

for(int count = 0; count < n - 1; count++)
{
int u = minDistance(dist, visited, n);

visited[u] = 1;

for(int v = 0; v < n; v++)
{
if(!visited[v] &&
graph[u][v] &&
dist[u] != INT_MAX &&
dist[u] + graph[u][v] < dist[v])
{
dist[v] = dist[u] + graph[u][v];
}
}
}

printf("\nShortest Distance from Source Vertex %d\n", source);
for(int i = 0; i < n; i++)
{
printf("Vertex %d --> Distance = %d\n", i, dist[i]);
}
}

int main()
{
int n, source;
int graph[MAX][MAX];

printf("Enter number of vertices: ");
```

```
scanf("%d", &n);

printf("Enter adjacency matrix:\n");
printf("(Enter 0 if no direct edge exists)\n");

for(int i = 0; i < n; i++)
{
for(int j = 0; j < n; j++)
{
scanf("%d", &graph[i][j]);
}
}

printf("Enter source vertex: ");
scanf("%d", &source);

dijkstra(graph, n, source);

return 0;
}
```

Output

```
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex: 0
Shortest Distance from Source Vertex 0
Vertex 0 --> Distance = 0
Vertex 1 --> Distance = 10
Vertex 2 --> Distance = 50
```

Vertex 3 --> Distance = 30

Vertex 4 --> Distance = 60

5 . **Topological Sorting**

```
#include <stdio.h>
#define MAX 100
int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int vertices;
void push(int vertex)
{
stack[++top] = vertex;
}
void DFS(int vertex)
{
visited[vertex] = 1;

for(int i = 0; i < vertices; i++)
{
if(graph[vertex][i] == 1 && !visited[i])
{
DFS(i);
}
}

push(vertex);
}

// Topological Sort Function
void topologicalSort()
{
for(int i = 0; i < vertices; i++)
```

```
{
visited[i] = 0;
}

for(int i = 0; i < vertices; i++)
{
if(!visited[i])
{
DFS(i);
}
}

printf("\nTopological Sorting Order:\n");

while(top != -1)
{
printf("%d ", stack[top--]);
}

printf("\n");
}

int main()
{
int edges;
int u, v;

printf("Enter number of vertices: ");
scanf("%d", &vertices);

printf("Enter number of edges: ");
scanf("%d", &edges);
```

```
// Initialize graph
for(int i = 0; i < vertices; i++)
{
for(int j = 0; j < vertices; j++)
{
graph[i][j] = 0;
}
}

printf("Enter directed edges (u v):\n");

for(int i = 0; i < edges; i++)
{
scanf("%d %d", &u, &v);
graph[u][v] = 1;
}
topologicalSort();
return 0;
}
```

Output

```
Enter number of vertices: 6
Enter number of edges: 6
Enter directed edges (u v):
5 2
5 0
4 0
4 1
2 3
3 1
```

Hashing Using Linear Probing

Code

```
#include <stdio.h>

#define EMPTY -1

int main()
{
int size, n;

printf("Enter hash table size: ");
scanf("%d", &size);

int hashTable[size];

// Initialize hash table
for(int i = 0; i < size; i++)
{
hashTable[i] = EMPTY;
}

printf("Enter number of keys: ");
scanf("%d", &n);

int key;

printf("Enter %d keys:\n", n);

for(int i = 0; i < n; i++)
{
scanf("%d", &key);

int index = key % size;
int originalIndex = index;

while(hashTable[index] != EMPTY)
{
index = (index + 1) % size;

if(index == originalIndex)
{
printf("Hash Table is Full!\n");
return 0;
}
}

hashTable[index] = key;
}

printf("\nFinal Hash Table:\n");
```

```
for(int i = 0; i < size; i++)
{
printf("Index %d : ", i);

if(hashTable[i] == EMPTY)
printf("EMPTY");
else
printf("%d", hashTable[i]);

printf("\n");
}

return 0;
}
```

Output

```
Enter hash table size: 7
Enter number of keys: 6
Enter 6 keys:
76
93
40
47
10
55
Final Hash Table:
Index 0 : 47
Index 1 : 55
Index 2 : 93
Index 3 : 10
Index 4 : EMPTY
Index 5 : 40
Index 6 : 76
```

Hashing Using Quadratic Probing

```
#include <stdio.h>

#define EMPTY -1

int main()
{
int size, n;

printf("Enter hash table size: ");
scanf("%d", &size);

int hashTable[size];

// Initialize hash table
for(int i = 0; i < size; i++)
{
hashTable[i] = EMPTY;
}

printf("Enter number of keys: ");
scanf("%d", &n);

printf("Enter the keys:\n");

for(int i = 0; i < n; i++)
{
int key;
scanf("%d", &key);

int index = key % size;
int j = 0;
```

```
while(hashTable[(index + j * j) % size] != EMPTY)
{
j++;
}

hashTable[(index + j * j) % size] = key;
}

printf("\nHash Table after insertion:\n");

for(int i = 0; i < size; i++)
{
printf("Index %d : ", i);

if(hashTable[i] == EMPTY)
printf("EMPTY");
else
printf("%d", hashTable[i]);

printf("\n");
}

return 0;
}
```

Output

```
Enter hash table size: 7
Enter number of keys: 3
Enter the keys:
22
30
50
Hash Table after insertion:
```

Index 0 : EMPTY

Index 1 : 22

Index 2 : 30

Index 3 : EMPTY

Index 4 : EMPTY

Index 5 : 50

Index 6 : EMPTY

8. **Insert the keys 27, 43, 69, 72 into a hash table of size 7, where**

- **$h_1(k) = k \bmod 7$**
- **$h_2(k) = 1 + (k \bmod 5)$**
- **using Double Hashing.**

Hashing Using Double Hashing

```
#include <stdio.h>

#define EMPTY -1

int main()
{
int size, n;

printf("Enter hash table size: ");
scanf("%d", &size);

int hashTable[size];

// Initialize hash table
for(int i = 0; i < size; i++)
{
hashTable[i] = EMPTY;
}

printf("Enter number of keys: ");
scanf("%d", &n);

printf("Enter the keys:\n");

for(int i = 0; i < n; i++)
{
int key;
scanf("%d", &key);
```

```
int h1 = key % size;
int h2 = 1 + (key % 5);

int j = 0;
int index;

do
{
index = (h1 + j * h2) % size;

if(hashTable[index] == EMPTY)
{
hashTable[index] = key;
break;
}

j++;

} while(j < size);

if(j == size)
{
printf("Hash Table is Full. Cannot insert %d\n", key);
}
}

printf("\nFinal Hash Table:\n");

for(int i = 0; i < size; i++)
{
printf("Index %d : ", i);
```

```
if(hashTable[i] == EMPTY)
printf("EMPTY");
else
printf("%d", hashTable[i]);

printf("\n");
}

return 0;
}
```

Output

```
Enter hash table size: 7
Enter number of keys: 4
Enter the keys:
27
43
69
72
Final Hash Table:
Index 0 : EMPTY
Index 1 : 43
Index 2 : 72
Index 3 : EMPTY
Index 4 : 69
Index 5 : EMPTY
Index 6 : 27
```

Quadratic Probing with User Input

Code

```
#include <stdio.h>

#define EMPTY -1

int main()
{
int size, n;

printf("Enter the size of hash table (prime number): ");
scanf("%d", &size);

int table[size];

// Initialize table
for(int i = 0; i < size; i++)
{
table[i] = EMPTY;
}

printf("Enter number of keys to insert: ");
scanf("%d", &n);

printf("Enter the keys:\n");

for(int i = 0; i < n; i++)
{
int key;
scanf("%d", &key);

int hash = key % size;
int j = 0;
int found = 0;

while(j < size)
{
int index = (hash + j * j) % size;

if(table[index] == EMPTY)
{
table[index] = key;
found = 1;
break;
}

j++;
}
```

```
if(!found)
{
printf("Cannot insert %d. Table is full.\n", key);
}
}

printf("\nHash Table:\n");

for(int i = 0; i < size; i++)
{
if(table[i] == EMPTY)
printf("Index %d : EMPTY\n", i);
else
printf("Index %d : %d\n", i, table[i]);
}

return 0;
}
```

Output

```
Enter the size of hash table (prime number): 7
Enter number of keys to insert: 4
Enter the keys:
22
30
50
44

Hash Table:

Index 0 : EMPTY
Index 1 : 22
Index 2 : 30
Index 3 : 44
Index 4 : EMPTY
Index 5 : 50
Index 6 : EMPTY
```

10. Implement a hash data structure and write initialize, search and insert routines for separate chaining hash table organization.

Hash Table Using Separate Chaining

```
#include <stdio.h>

#include <stdlib.h>


#define SIZE 10


// Structure for Linked List Node

struct Node

{

int data;

struct Node *next;

};


// Hash Table

struct Node *hashTable[SIZE];


// Initialize Hash Table

void initialize()

{

for(int i = 0; i < SIZE; i++)

{

hashTable[i] = NULL;

}

}


// Hash Function

int hashFunction(int key)

{

return key % SIZE;

}
```

```
// Insert Key
void insert(int key)
{
int index = hashFunction(key);

struct Node *newNode =
(struct Node *)malloc(sizeof(struct Node));

newNode->data = key;
newNode->next = hashTable[index];

hashTable[index] = newNode;

printf("%d inserted successfully.\n", key);
}

// Search Key
void search(int key)
{
int index = hashFunction(key);

struct Node *temp = hashTable[index];

while(temp != NULL)
{
if(temp->data == key)
{
printf("%d found at index %d\n",
key, index);
return;
}

temp = temp->next;
```

```
}

printf("%d not found in the hash table.\n",
key);
}

// Display Hash Table
void display()
{
printf("\nHash Table:\n");

for(int i = 0; i < SIZE; i++)
{
printf("[%d] --> ", i);

struct Node *temp = hashTable[i];

while(temp != NULL)
{
printf("%d -> ", temp->data);
temp = temp->next;
}

printf("NULL\n");
}
}

int main()
{
int choice, key;

initialize();
```

```
while(1)
{
printf("\n------ HASH TABLE MENU ------\n");
printf("1. Insert\n");
printf("2. Search\n");
printf("3. Display\n");
printf("4. Exit\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
case 1:
printf("Enter key to insert: ");
scanf("%d", &key);
insert(key);
break;

case 2:
printf("Enter key to search: ");
scanf("%d", &key);
search(key);
break;

case 3:
display();
break;

case 4:
exit(0);

default:
```

```
printf("Invalid Choice!\n");
}
}

return 0;
}
```

Output

```
1. Insert
2. Search
3. Display
4. Exit

Enter your choice: 1
Enter key to insert: 25
25 inserted successfully.

Enter your choice: 1
Enter key to insert: 35
35 inserted successfully.

Enter your choice: 1
Enter key to insert: 15
15 inserted successfully.

Enter your choice: 3
Hash Table:

[0] --> NULL
[1] --> NULL
[2] --> NULL
[3] --> NULL
[4] --> NULL
[5] --> 15 -> 35 -> 25 -> NULL
```

[6] --> NULL

[7] --> NULL

[8] --> NULL

[9] --> NULL